ThreatMatrix AI — Master Documentation v1.0

Part 5: Development Timeline, Team Workflow & Deployment Guide

> **Part:** 5 of 5 | **Version:** 1.0.0 | **Date:** 2026-02-23
> **Prev:** Part 4 — ML & LLM | **Start:** Part 1 — Strategy

Table of Contents — Part 5

1. Development Methodology
2. Project Structure & Monorepo Layout
3. Week-by-Week Development Plan
4. Task Assignment Matrix
5. Version & Iteration Strategy
6. Deployment Guide
7. Testing Strategy
8. Demo Day Preparation
9. Documentation Deliverables
10. Quick Reference & Commands

---

1. Development Methodology

1.1 Workflow: Aggressive Agile Rapid Prototyping

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    DEVELOPMENT WORKFLOW                                  │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  Sprint Cycle: 1 WEEK (8 sprints total)                                  │
│                                                                          │
│  Monday:    Sprint planning (30 min) → Define week's deliverables       │
│  Mon-Sat:   Build, build, build (each iteration is functional MVP)      │
│  Sunday:    Integration test → demo internal → retrospective            │
│                                                                          │
│  Rule #1: Every Sunday, the system must be DEMOABLE                      │
│  Rule #2: No feature is "done" until it's visible in the UI              │
│  Rule #3: If it's not in this document, it's out of scope                │
│  Rule #4: Version aggressively — tag every working state                 │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

1.2 Git Workflow

Branch	Purpose	Merge Strategy

Branch	Purpose	Merge Strategy
`main`	Production-ready, always deployable	Protected, PR only
`develop`	Integration branch	Merge from feature branches
`feature/*`	Individual features	`feature/war-room-map`, `feature/ml-pipeline`
`hotfix/*`	Critical fixes	Merge to main + develop

Versioning: Semantic — `v0.1.0` (first MVP) → `v0.2.0` → ... → `v1.0.0` (final submission)

1.3 Communication

Channel	Purpose	Frequency
Git commits	Technical progress tracking	Every meaningful change
WhatsApp Group	Quick coordination, blockers	Real-time
Weekly Demo	Show working software	Every Sunday
This Document	Single source of truth	Updated as decisions change

---

2. Project Structure & Monorepo Layout

2.1 Repository Structure

```
threatmatrix-ai/
├── docs/                              # This documentation
│   ├── MASTER_DOC_PART1_STRATEGY.md
│   ├── MASTER_DOC_PART2_ARCHITECTURE.md
│   ├── MASTER_DOC_PART3_MODULES.md
│   ├── MASTER_DOC_PART4_ML_LLM.md
│   └── MASTER_DOC_PART5_TIMELINE.md
│
├── backend/                           # Python FastAPI Backend
│   ├── app/
│   │   ├── main.py                    # FastAPI application entry
│   │   ├── config.py                  # Environment configuration
│   │   ├── database.py                # PostgreSQL connection + models
│   │   ├── dependencies.py            # Auth dependencies
│   │   ├── api/
│   │   │   ├── v1/
│   │   │   │   ├── auth.py            # Authentication endpoints
│   │   │   │   ├── flows.py           # Flow data endpoints
│   │   │   │   ├── alerts.py          # Alert management endpoints
│   │   │   │   ├── ml.py              # ML model endpoints
│   │   │   │   ├── intel.py           # Threat intelligence endpoints
│   │   │   │   ├── llm.py             # LLM chat/analysis endpoints
│   │   │   │   ├── capture.py         # Capture engine control
│   │   │   │   ├── reports.py         # Report generation endpoints
│   │   │   │   └── system.py          # System health/config
```

```
│   │   │   └── websocket.py          # WebSocket event broadcasting
│   │   ├── models/                   # SQLAlchemy/Pydantic models
│   │   │   ├── user.py
│   │   │   ├── flow.py
│   │   │   ├── alert.py
│   │   │   ├── intel.py
│   │   │   └── ml_model.py
│   │   ├── services/                 # Business logic layer
│   │   │   ├── auth_service.py
│   │   │   ├── flow_service.py
│   │   │   ├── alert_service.py
│   │   │   ├── ml_service.py
│   │   │   ├── intel_service.py
│   │   │   ├── llm_gateway.py        # Multi-provider LLM router
│   │   │   └── report_service.py
│   │   └── schemas/                  # Pydantic request/response schemas
│   │       ├── auth.py
│   │       ├── flow.py
│   │       ├── alert.py
│   │       └── common.py
│   ├── capture/                      # Packet capture engine
│   │   ├── engine.py                 # Main capture loop (Scapy)
│   │   ├── flow_aggregator.py        # Flow assembly from packets
│   │   ├── feature_extractor.py      # 40+ feature computation
│   │   └── pcap_processor.py         # PCAP file ingestion
│   ├── ml/                           # Machine learning pipeline
│   │   ├── datasets/                 # Dataset loaders
│   │   │   ├── nsl_kdd.py
│   │   │   └── cicids2017.py
│   │   ├── models/                   # Model implementations
│   │   │   ├── isolation_forest.py
│   │   │   ├── random_forest.py
│   │   │   └── autoencoder.py
│   │   ├── training/                 # Training scripts
│   │   │   ├── train_all.py          # Train all models
│   │   │   ├── evaluate.py           # Evaluation metrics
│   │   │   └── hyperparams.py        # Hyperparameter configs
│   │   ├── inference/                # Real-time inference
│   │   │   ├── model_manager.py      # Load and manage models
│   │   │   ├── ensemble_scorer.py    # Composite scoring
│   │   │   └── worker.py            # Redis subscriber + inference loop
│   │   └── saved_models/            # Serialized trained models
│   ├── requirements.txt
│   ├── Dockerfile
│   └── alembic/                      # Database migrations
│       └── versions/
│
├── frontend/                         # Next.js Command Center
│   ├── app/
│   │   ├── layout.tsx                # Root layout (dark theme, sidebar)
│   │   ├── page.tsx                  # Redirect to /war-room
│   │   ├── war-room/
│   │   │   └── page.tsx
│   │   ├── hunt/
```

```
│   │   │   └── page.tsx
│   │   ├── intel/
│   │   │   └── page.tsx
│   │   ├── network/
│   │   │   └── page.tsx
│   │   ├── ai-analyst/
│   │   │   └── page.tsx
│   │   ├── alerts/
│   │   │   └── page.tsx
│   │   ├── forensics/
│   │   │   └── page.tsx
│   │   ├── ml-ops/
│   │   │   └── page.tsx
│   │   ├── reports/
│   │   │   └── page.tsx
│   │   └── admin/
│   │       ├── page.tsx
│   │       ├── users/page.tsx
│   │       ├── config/page.tsx
│   │       ├── llm-budget/page.tsx
│   │       └── audit/page.tsx
│   ├── components/
│   │   ├── layout/
│   │   │   ├── Sidebar.tsx            # Icon-only sidebar navigation
│   │   │   ├── TopBar.tsx             # Threat level, notifications, user
│   │   │   └── StatusBar.tsx          # System status footer
│   │   ├── war-room/
│   │   │   ├── ThreatMap.tsx          # Deck.gl + Maplibre globe
│   │   │   ├── ThreatGauge.tsx        # DEFCON-style level indicator
│   │   │   ├── LiveAlertFeed.tsx      # Scrolling alert ticker
│   │   │   ├── MetricCard.tsx         # Animated metric display
│   │   │   ├── ProtocolChart.tsx      # Protocol distribution donut
│   │   │   ├── TrafficTimeline.tsx    # Area chart with anomaly overlay
│   │   │   ├── TopTalkers.tsx         # IP volume ranking
│   │   │   ├── GeoDistribution.tsx    # Country breakdown
│   │   │   └── AIBriefingWidget.tsx   # LLM threat summary
│   │   ├── ai-analyst/
│   │   │   ├── ChatInterface.tsx      # Terminal-style chat
│   │   │   ├── QuickActions.tsx       # Preset action buttons
│   │   │   └── ContextPanel.tsx       # Data reference panel
│   │   ├── alerts/
│   │   │   ├── AlertTable.tsx
│   │   │   ├── AlertDetail.tsx
│   │   │   └── AlertTimeline.tsx
│   │   ├── ml-ops/
│   │   │   ├── ModelComparison.tsx
│   │   │   ├── ConfusionMatrix.tsx
│   │   │   ├── ROCCurve.tsx
│   │   │   └── FeatureImportance.tsx
│   │   └── shared/
│   │       ├── GlassPanel.tsx         # Glassmorphism container
│   │       ├── DataTable.tsx          # Reusable sortable table
│   │       ├── StatusBadge.tsx        # Severity/status badges
│   │       └── LoadingState.tsx       # Skeleton loaders
```

```
│   ├── hooks/
│   │   ├── useWebSocket.ts          # WebSocket connection manager
│   │   ├── useFlows.ts              # Flow data queries
│   │   ├── useAlerts.ts             # Alert queries + mutations
│   │   ├── useMLModels.ts           # ML model data
│   │   └── useLLM.ts                # LLM chat with streaming
│   ├── lib/
│   │   ├── api.ts                   # API client (fetch wrapper)
│   │   ├── websocket.ts             # WebSocket client
│   │   ├── constants.ts             # Colors, thresholds, config
│   │   └── utils.ts                 # Formatting, helpers
│   ├── styles/
│   │   └── globals.css              # Design system (CSS variables)
│   ├── messages/
│   │   ├── en.json                  # English translations
│   │   └── am.json                  # Amharic translations
│   ├── next.config.ts
│   ├── package.json
│   └── Dockerfile
│
├── docker-compose.yml                # Full stack orchestration
├── docker-compose.dev.yml            # Development overrides
├── nginx/
│   └── nginx.conf                    # Reverse proxy config
├── .env.example                      # Environment template
├── README.md                         # Quick start guide
└── Makefile                          # Common commands
```

---

3. Week-by-Week Development Plan

Week 1: Foundation (Feb 24 — Mar 2)

Task	Owner	Deliverable
Project init: monorepo, Docker Compose, CI	Lead	Running dev environment
PostgreSQL schema + Alembic migrations	Lead	All tables created
FastAPI skeleton: auth, health, CORS, OpenAPI	Lead	`/docs` showing all endpoints
Next.js 16 init: layout, sidebar, theme, CSS vars	Full-Stack	Dark War Room shell with navigation
Redis setup + pub/sub test	Lead	Message passing verified
Design system: colors, typography, GlassPanel component	Full-Stack	`globals.css` + shared components
Legal framework for VPS traffic capture	Business Mgr	Written authorization document
Dev environment setup, test plan draft	Tester	Test plan document

End of Week 1 Demo: Empty but beautiful dark dashboard shell, API docs visible at `/docs`, database tables exist.

Week 2: Capture + Core UI (Mar 3 — Mar 9)

Task	Owner	Deliverable
Scapy capture engine: packet sniffing, flow aggregation	Lead	Capturing live VPS traffic
Feature extraction pipeline (40+ features)	Lead	Feature vectors in PostgreSQL
Redis pub/sub integration (capture → Redis → API)	Lead	Real-time flow publishing
War Room layout: grid structure, all component shells	Full-Stack	War Room page with placeholder data
MetricCard, StatusBadge, DataTable components	Full-Stack	Reusable UI components
WebSocket client hook + connection manager	Full-Stack	`useWebSocket.ts` working
Market research document	Business Mgr	Competitive analysis report
Test data generation scripts	Tester	Sample flow/alert data

End of Week 2 Demo: Live traffic being captured and stored. War Room layout visible with placeholder/mock data.

Week 3: ML Pipeline (Mar 10 — Mar 16)

Task	Owner	Deliverable
Download + prepare NSL-KDD dataset	Lead	Cleaned, split, normalized data
Train Isolation Forest	Lead	Model saved, metrics computed
Train Random Forest	Lead	Model saved, metrics computed
Train Autoencoder (TensorFlow)	Lead	Model saved, metrics computed
Ensemble scorer implementation	Lead	Composite scoring working
Model evaluation framework (all metrics)	Lead	Comparison table generated
War Room: ThreatMap (Deck.gl + Maplibre)	Full-Stack	Interactive dark world map
War Room: TrafficTimeline, ProtocolChart	Full-Stack	Live-updating charts
Network Flow module: basic layout	Full-Stack	Traffic analysis page
Competitor analysis report	Business Mgr	Completed document
Dataset validation testing	Tester	Verified data integrity

End of Week 3 Demo: ML models trained with real metrics. War Room showing live map and charts. Traffic being captured and scored.

Week 4: Intelligence Integration (Mar 17 — Mar 23)

Task	Owner	Deliverable
LLM Gateway service (multi-provider)	Lead	DeepSeek + GLM + Groq routing
AI Analyst backend: chat, briefing, analysis endpoints	Lead	`/api/v1/llm/*` endpoints working
Threat intel aggregator (OTX + AbuseIPDB)	Lead	IOCs syncing and stored
Real-time inference pipeline (ML Worker)	Lead	Live scoring of captured traffic
Alert engine: auto-create alerts from anomalies	Lead	Alerts appearing in PostgreSQL
AI Analyst chat UI (streaming responses)	Full-Stack	Chat interface with typing effect
Intel Hub: IOC browser, IP lookup	Full-Stack	Threat intelligence page
Alert Console: table, detail drawer	Full-Stack	Alert listing working
User personas, use case docs	Business Mgr	Completed documents
API testing (Postman/curl)	Tester	API test suite

End of Week 4 Demo: Full intelligence loop working — capture → ML scoring → alerts → LLM narratives → UI display. AI Analyst answering questions. **This is the critical MVP milestone.**

Week 5: Feature Depth (Mar 24 — Mar 30)

Task	Owner	Deliverable
Anomaly scoring refinement + threshold tuning	Lead	Reduced false positives
PCAP upload + analysis pipeline	Lead	Upload and analyze historical traffic
WebSocket alert broadcasting	Lead	Real-time alerts in browser
CICIDS2017 dataset training (secondary validation)	Lead	Models validated on second dataset
Forensics Lab UI: PCAP upload, results	Full-Stack	Working forensics page
ML Ops dashboard: confusion matrix, ROC curves	Full-Stack	Model performance visualizations
War Room: LiveAlertFeed, TopTalkers, GeoDistribution	Full-Stack	All War Room widgets populated
Threat Hunt: query builder UI	Full-Stack	Basic hunt functionality
Demo script draft, slide deck start	Business Mgr	First draft of presentation
Integration testing	Tester	End-to-end test results

End of Week 5 Demo: Feature-complete War Room. Forensics working. ML dashboards with real metrics.

Week 6: Reports + Enterprise (Mar 31 — Apr 6)

Task	Owner	Deliverable
PDF report generation (ReportLab)	Lead	Daily threat summary PDF
RBAC enforcement on all endpoints	Lead	Role-based access verified
LLM budget tracking + caching	Lead	Budget dashboard data
System health monitoring endpoint	Lead	`/system/health` complete
Reports module UI: generate, list, download	Full-Stack	Report generation page
Administration: user management, config	Full-Stack	Admin pages working
LLM Budget dashboard UI	Full-Stack	Token usage visualization
Network Flow: connection graph, bandwidth	Full-Stack	Enhanced network analysis
Revenue projection model	Business Mgr	Financial projections
End-to-end testing	Tester	Full system test report

End of Week 6 Demo: Enterprise features complete. Reports generating. Admin panel working.

Week 7: Polish + i18n (Apr 7 — Apr 13)

Task	Owner	Deliverable
Performance optimization: query tuning, indexing	Lead	<200ms API responses
Model tuning: retrain with optimized hyperparameters	Lead	Improved accuracy metrics
Pre-built PCAP demo scenarios (DDoS, scan, C2)	Lead	3-5 attack scenarios ready
Attack simulation scripts (nmap, hping3 against VPS)	Lead	Real anomalies in live traffic
Responsive design polish	Full-Stack	Mobile & tablet layouts
Animations: page transitions, micro-interactions	Full-Stack	Framer Motion throughout
Loading states, error boundaries, empty states	Full-Stack	Polished edge cases
Amharic/English i18n implementation	Full-Stack + Tester	Language toggle working
Final business plan, presentation slides	Business Mgr	Complete slide deck
UAT testing, Amharic translation review	Tester	UAT report + translations

End of Week 7 Demo: Polished, animated, bilingual system. Demo scenarios prepared.

Week 8: Final Push (Apr 14 — Apr 20)

Task	Owner	Deliverable
Production deployment on VPS	Lead	Live system running 24/7
SSL (Let's Encrypt), domain config	Lead	HTTPS enabled
Security hardening: rate limiting, input validation	Lead	Penetration test passed
API documentation finalization (Swagger)	Lead	Complete API reference
Final UI bug fixes and polish	Full-Stack	Pixel-perfect War Room
Dark/light theme toggle (stretch)	Full-Stack	Theme switching
Final presentation prep	Business Mgr	Presentation rehearsal
User manual document	Business Mgr	User guide PDF
Final QA, demo day rehearsal	Tester	Go/no-go decision
Team rehearsal: live demo walkthrough	All	Smooth 20-min demo

End of Week 8: SHIP IT. 🚀

---

4. Task Assignment Matrix

4.1 Feature Ownership

Feature	Lead Architect	Full-Stack Dev	Business Mgr	Tester
FastAPI Backend	■			
Database Schema	■			
Capture Engine	■			
ML Pipeline	■			
LLM Gateway	■			
Alert Engine	■			
WebSocket Server	■			
War Room UI	■	■		
All Other UI		■		
Design System	■	■		
i18n Setup		■		■
Amharic Translations				■
Business Documents			■	
Presentation			■	

Feature	Lead Architect	Full-Stack Dev	Business Mgr	Tester
Market Research			■■■■	
Test Plan				■■■■
Testing Execution				■■■■
Attack Simulation	■■■			■
Documentation	■■■		■	
Deployment	■■■■			

4.2 Estimated Effort Distribution

Team Member	Hours/Week	Total Hours (8 weeks)	Focus
Lead Architect	40-60	320-480	Backend, ML, LLM, Architecture
Full-Stack Dev	30-40	240-320	Frontend, UI/UX, Charts
Business Manager	15-20	120-160	Docs, Research, Presentation
Tester	10-15	80-120	Testing, Translations, QA

---

5. Version & Iteration Strategy

5.1 Version Milestones

Version	Date	Content	Status
`v0.1.0`	Week 1	Project skeleton, DB, auth, UI shell	Foundation
`v0.2.0`	Week 2	Capture engine, flow storage, War Room layout	Data pipeline
`v0.3.0`	Week 3	ML models trained, basic scoring, map + charts	ML integration
`v0.4.0`	Week 4	LLM integration, AI Analyst, threat intel, alerts	**Critical MVP** ☑
`v0.5.0`	Week 5	PCAP forensics, ML dashboards, full War Room	Feature depth
`v0.6.0`	Week 6	Reports, admin, RBAC, budget tracking	Enterprise features
`v0.7.0`	Week 7	Polish, animations, i18n, demo scenarios	Pre-release
`v1.0.0`	Week 8	Production deployment, final fixes, documentation	**Release** 🚀

5.2 Each Version is Demoable

Every version tagged in Git represents a **fully functional system**. If development stops at any version, the project is still presentable:

- **v0.4.0 minimum:** Live capture + ML scoring + AI chat + alerts = complete senior project
- **v0.6.0 ideal:** All of above + reports + admin + forensics = enterprise product

- **v1.0.0 target:** Fully polished, deployed, documented = industry-grade

---

6. Deployment Guide

6.1 Development Environment Setup

```
# 1. Clone repository
git clone https://github.com/kidusabdula/threatmatrix-ai.git
cd threatmatrix-ai

# 2. Backend setup
cd backend
python -m venv venv
source venv/bin/activate  # or venv\Scripts\activate on Windows
pip install -r requirements.txt

# 3. Frontend setup
cd ../frontend
npm install

# 4. Start infrastructure (PostgreSQL + Redis)
docker-compose -f docker-compose.dev.yml up -d postgres redis

# 5. Run database migrations
cd ../backend
alembic upgrade head

# 6. Start backend
uvicorn app.main:app --reload --port 8000

# 7. Start frontend (new terminal)
cd ../frontend
npm run dev

# 8. Access
# Frontend: http://localhost:3000
# Backend API: http://localhost:8000
# API Docs: http://localhost:8000/docs
```

6.2 Production Deployment (VPS)

```
# 1. SSH into VPS
ssh user@your-vps-ip

# 2. Clone and configure
git clone https://github.com/kidusabdula/threatmatrix-ai.git
cd threatmatrix-ai
cp .env.example .env
# Edit .env with production values
```

```
# 3. Deploy with Docker Compose
docker-compose up -d --build

# 4. Run migrations
docker-compose exec backend alembic upgrade head

# 5. Create admin user
docker-compose exec backend python -m app.scripts.create_admin

# 6. Start capture engine (requires root for raw sockets)
docker-compose exec --privileged capture python -m capture.engine

# 7. Verify
curl https://your-domain.com/api/v1/system/health
```

6.3 Docker Compose Configuration

```
# docker-compose.yml
version: "3.8"

services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: threatmatrix
      POSTGRES_USER: threatmatrix
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pg_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U threatmatrix"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    command: redis-server --maxmemory 256mb --maxmemory-policy allkeys-lru

  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
```

```
      DATABASE_URL:
postgresql://threatmatrix:${DB_PASSWORD}@postgres:5432/threatmatrix
      REDIS_URL: redis://redis:6379
      DEEPSEEK_API_KEY: ${DEEPSEEK_API_KEY}
      GLM_API_KEY: ${GLM_API_KEY}
      GROQ_API_KEY: ${GROQ_API_KEY}
      JWT_SECRET: ${JWT_SECRET}
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_started
    volumes:
      - ./backend/ml/saved_models:/app/ml/saved_models
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4

  capture:
    build: ./backend
    network_mode: host
    privileged: true
    environment:
      REDIS_URL: redis://localhost:6379
      DATABASE_URL:
postgresql://threatmatrix:${DB_PASSWORD}@localhost:5432/threatmatrix
      CAPTURE_INTERFACE: ${CAPTURE_INTERFACE:-eth0}
    volumes:
      - ./pcaps:/app/pcaps
    command: python -m capture.engine
    depends_on:
      - redis
      - postgres

  ml-worker:
    build: ./backend
    environment:
      DATABASE_URL:
postgresql://threatmatrix:${DB_PASSWORD}@postgres:5432/threatmatrix
      REDIS_URL: redis://redis:6379
    volumes:
      - ./backend/ml/saved_models:/app/ml/saved_models
    command: python -m ml.inference.worker
    depends_on:
      - redis
      - postgres
      - backend

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf
      - ./certs:/etc/nginx/certs
```

```
    depends_on:
      - backend

volumes:
  pg_data:
  redis_data:
```

6.4 Environment Variables

```
# .env.example

# Database
DB_PASSWORD=your_secure_password_here

# JWT
JWT_SECRET=your_jwt_secret_key_here
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=15

# LLM Providers
DEEPSEEK_API_KEY=sk-...
GLM_API_KEY=...
GROQ_API_KEY=gsk_...
LLM_MONTHLY_BUDGET_USD=50.00

# Capture
CAPTURE_INTERFACE=eth0
CAPTURE_BPF_FILTER=

# Threat Intel
OTX_API_KEY=...
ABUSEIPDB_API_KEY=...
VIRUSTOTAL_API_KEY=...

# Redis
REDIS_URL=redis://redis:6379

# Frontend
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_WS_URL=ws://localhost:8000/ws
```

---

7. Testing Strategy

7.1 Test Layers

Layer	Tool	Scope	Owner
Unit Tests	pytest	Backend services, feature extraction, scoring	Lead

Layer	Tool	Scope	Owner
API Tests	pytest + httpx	All REST endpoints	Lead + Tester
ML Tests	pytest	Model accuracy thresholds, inference correctness	Lead
Frontend Tests	Jest + React Testing Library	Component rendering, hooks	Full-Stack
Integration Tests	Docker Compose	End-to-end capture → alert → UI	Tester
UAT	Manual, scripted scenarios	Full user workflows	Tester
Performance	locust or k6	API throughput, WebSocket load	Lead

7.2 ML Validation Criteria

Model	Minimum Accuracy	Minimum F1	Maximum Inference Time
Isolation Forest	88%	0.86	2ms
Random Forest	94%	0.93	3ms
Autoencoder	91%	0.90	10ms
Ensemble	95%	0.94	15ms

7.3 Demo Scenario Test Cases

Scenario	Attack Tool	Expected Detection	Alert Severity
Port Scan	`nmap -sS target_ip`	Probe detected, port sweep classification	High
DDoS Simulation	`hping3 --flood target_ip`	Volume anomaly, DDoS classification	Critical
DNS Tunneling	`iodine` or crafted DNS queries	Unusual DNS pattern, high entropy	Medium
Brute Force SSH	`hydra -l root -P wordlist ssh://target`	Failed login volume spike	High
Normal Traffic	Regular browsing, API calls	No false alerts	—

8. Demo Day Preparation

8.1 Demo Script (20 minutes)

Time	Section	What to Show	Who Presents
0:00-0:30	**Hook**	Open War Room. Live data flowing. Map active.	Lead
0:30-2:30	**Problem**	Ethiopia's cyber gap. Market stats. Why now.	Business Mgr
2:30-5:30	**Architecture**	Three-tier diagram. Tech stack. ML strategy.	Lead
5:30-10:30	**Live Demo**	War Room → run nmap attack → alert fires → AI explains	Lead
10:30-13:00	**AI Analyst**	Ask AI about the attack. Generate briefing. Amharic translation.	Lead
13:00-15:00	**ML Results**	Model comparison table. Confusion matrices. ROC curves.	Lead
15:00-17:00	**Enterprise**	Report generation (PDF). Admin panel. User management.	Full-Stack
17:00-19:00	**Business Case**	Pricing tiers. Target customers. Revenue model.	Business Mgr
19:00-20:00	**Close**	Future roadmap. "This is v1.0." Thank you.	Lead

8.2 Backup Plans

Risk	Backup
VPS is down	Pre-recorded demo video (record during Week 7)
LLM API is down	Pre-cached responses for demo scenarios
No live traffic	Pre-loaded PCAP with interesting anomalies
Browser crashes	Second laptop with same setup
Internet fails	Full system runs locally on Docker (offline capable)

8.3 Pre-Demo Checklist

- ☐ VPS running and capture active for 24+ hours (accumulate data)
- ☐ All 3 ML models loaded and inference working
- ☐ LLM APIs responding (test all 3 providers)
- ☐ Threat intel feeds synced in last 6 hours
- ☐ Demo attack scenarios tested and working
- ☐ PDF report generated successfully
- ☐ Amharic translations loaded
- ☐ Demo user account created (analyst role)
- ☐ Backup video recorded
- ☐ Slide deck exported to PDF (offline)

- ☐ Second laptop charged and configured

---

9. Documentation Deliverables

9.1 Required Documents

Document	Owner	Format	Purpose
Master Documentation (this)	Lead	5 MD files	Single source of truth
User Manual	Business Mgr	PDF	End-user guide for operating ThreatMatrix
API Reference	Auto-generated	Swagger/OpenAPI	Developer documentation
ML Evaluation Report	Lead	PDF/MD	Academic metrics, model comparison
Business Plan	Business Mgr	PDF	Market analysis, monetization, projections
Presentation Slides	Business Mgr	PPT/PDF	Demo day presentation
Test Report	Tester	PDF/MD	Test results, coverage, issues
Deployment Guide	Lead	MD	How to deploy (Section 6 of this doc)

9.2 Academic Submission Package

```
submission/
├── ThreatMatrix_AI_Report.pdf          # Main academic report
├── ThreatMatrix_AI_Presentation.pptx   # Slide deck
├── ThreatMatrix_AI_User_Manual.pdf     # User guide
├── source_code/                        # Full repository
│   └── threatmatrix-ai/
├── demo_video.mp4                      # Recorded demo (backup)
└── appendices/
    ├── ML_Evaluation_Report.pdf
    ├── API_Documentation.pdf
    ├── Business_Plan.pdf
    └── Test_Report.pdf
```

---

10. Quick Reference & Commands

10.1 Common Commands

```
# Development
make dev                    # Start all services in dev mode
make test                   # Run all tests
make lint                   # Lint Python + TypeScript
make migrate                # Run database migrations
make train-models           # Train all ML models

# Docker
docker-compose up -d        # Start production stack
docker-compose logs -f      # Follow logs
docker-compose ps           # Check service status
docker-compose down         # Stop all services

# Backend
cd backend
uvicorn app.main:app --reload                    # Dev server
pytest tests/ -v                                  # Run tests
alembic upgrade head                              # Apply migrations
alembic revision --autogenerate -m "description"  # Create migration

# Frontend
cd frontend
npm run dev                 # Dev server on :3000
npm run build               # Production build
npm run lint                # ESLint check

# ML
cd backend
python -m ml.training.train_all                   # Train all models
python -m ml.training.evaluate                    # Generate evaluation report
python -m ml.inference.worker                     # Start inference worker

# Capture
sudo python -m capture.engine                     # Start capture (needs root)
python -m capture.pcap_processor file.pcap        # Process PCAP file
```

10.2 Key URLs

Service	URL	Credentials
Frontend (dev)	http://localhost:3000	Login with created user
Backend API (dev)	http://localhost:8000	—
API Documentation	http://localhost:8000/docs	—
API Redoc	http://localhost:8000/redoc	—
Production Frontend	https://threatmatrix-ai.vercel.app	—
Production API	https://api.threatmatrix-ai.com	—
PostgreSQL	localhost:5432	threatmatrix / (password)

Service	URL	Credentials
Redis	localhost:6379	—

10.3 Dataset Downloads

Dataset	URL	Size
NSL-KDD	https://www.unb.ca/cic/datasets/nsl.html	~25 MB
CICIDS2017	https://www.unb.ca/cic/datasets/ids-2017.html	~6.5 GB
UNSW-NB15	https://research.unsw.edu.au/projects/unsw-nb15-dataset	~1.6 GB

10.4 API Key Registration

Service	Registration URL	Free Tier
AlienVault OTX	https://otx.alienvault.com/	Unlimited
AbuseIPDB	https://www.abuseipdb.com/register	1,000/day
VirusTotal	https://www.virustotal.com/gui/join-us	500/day
DeepSeek	https://platform.deepseek.com/	$5 free credits
Groq	https://console.groq.com/	Free tier available
GLM (Zhipu)	https://open.bigmodel.cn/	Free tier available

Final Notes

Critical Success Factors

1. **Week 4 is the make-or-break moment** — if capture + ML + LLM + alerts are working end-to-end, success is virtually guaranteed
2. **War Room is the demo killer** — allocate disproportionate design effort here
3. **ML metrics are the academic shield** — advisors can't argue with precision/recall/F1 numbers
4. **AI Analyst is the innovation story** — "we integrated LLM-powered threat analysis" is the differentiator
5. **Document everything** — this 5-part document IS the proof of engineering maturity

What NOT To Do

- ❌ Don't add features not in this document (scope creep kills projects)
- ❌ Don't spend time on Kafka, Elasticsearch, or Kubernetes (overkill)
- ❌ Don't try to capture traffic on networks you don't own (legal risk)
- ❌ Don't build your own threat intel database from scratch (use feeds)
- ❌ Don't attempt real-time ML training (batch is fine, retrain weekly)
- ❌ Don't sacrifice UI polish for features (fewer polished features > many rough ones)
- ❌ Don't skip the demo rehearsal (practice the 20-min walkthrough 3+ times)

> **End of Part 5** — Return to Part 1: Executive Strategy for complete navigation.

Master Document Index

Part	Title	Key Contents
Part 1	Executive Strategy	Business case, market analysis, advisor strategy, risk assessment
Part 2	Architecture	Three-tier system, DB schema, API design, real-time strategy, security
Part 3	Modules & UI/UX	All 10 modules specified, design system, War Room blueprint
Part 4	ML & LLM	3 ML models, training config, LLM gateway, prompt templates, data strategy
Part 5	Timeline & Deployment	Week-by-week plan, task matrix, Docker deployment, testing, demo prep

---

ThreatMatrix AI Master Documentation v1.0 — Part 5 of 5
© 2026 ThreatMatrix AI. All rights reserved.
Built with conviction. Shipped with precision.